

OpenContext Database

Files: All information originates from three parquet files: `oc_all_manifest.parquet`, `oc_all_assertions.parquet`, and `oc_all_resources.parquet`. These files collectively encode a graph-based knowledge system instead of a relational table.

1. **oc_all_manifest.parquet:** This file gives the object information. It showed that every row represents a single entity identified by a UUID, and that entities are differentiated only by an `item_type` field. Artifacts are not a special table; they are simply entities whose `item_type` is "`subjects`". Media objects, contexts, projects, sites, predicates, classes, and vocabularies all live in the same file and share the same structural representation. Artifact rows do contain identifiers, labels, class UUIDs, project UUIDs, and a `context_uuid`, but no guaranteed semantic values such as coordinates, dates, or image references.
2. **oc_all_assertions.parquet:** All actual meaning lives in this file. It encodes all semantic information as directed edges. Each assertion links a `subject_uuid` to either an `object_uuid` or a literal value via a `predicate_uuid`. Predicates themselves are entities defined in the manifest file. So things like location, dates, and image relationships are all assertions, not artifact columns.
3. **oc_all_resources.parquet:** For images, joining artifacts directly to `oc_all_resources.parquet` doesn't work because resources don't point to artifacts. Each resource points to a **media UUID** via `item_uuid`. Media entities (conceptual images) live in the manifest (`item_type = "media"`). Artifacts connect to media through assertions, and media connect to actual files through resources. So the only valid path is: artifact → assertion → media → resource. Using this gave ~2.1M total subjects → ~68k with images → **33,412 artifacts** after filtering for image + location signal + temporal signal. Link: [📁 metadata](#)

(NOTE: Boolean indicators such as `has_location` and `has_date` were generated as summaries of graph connectivity and are not representations of stored values. This was done by iterating over artifact UUIDs and scanning assertions to detect the existence of location-related and date-related predicates)

Babel Dataset: After I had the artifact-to-image mapping, I generated an `img2dataset` TSV and started image downloads (around **385k images**) on a compute node, current status is around 115k images before the connection got closed. The download settings are max dimension 1024 pixels and aspect ratio preserved. I also did a sanity-check workflow: copied a small sample to the login node for inspection. Link: [📁 samples](#)

4. **Location data:** The manifest does not contain latitude or longitude fields for artifacts. All location information is in assertions whose predicates correspond to geographic

concepts. Some assertions attach directly to artifact UUIDs and contain numeric coordinate values (latitude and longitude). I enumerated location-related predicates (about **290**) by parsing multiple coordinate formats: numeric lat/lon, approximate coords, EPSG coords, and coordinate strings. Some artifacts have location data attached to context entities which are referenced by artifacts via `context_uuid`. Here, artifacts inherit location information from their contexts. Many location assertions contain only descriptive or relative text rather than numeric coordinates. I split the source of coordinates into two cases: direct coordinates attached to the artifact itself vs coordinates attached to a context entity that the artifact inherits via `context_uuid`. The current numeric-coordinate coverage is: **~1,271 artifacts direct, ~1,262 inherited, ~2,533 total with best available lat/lon,**

5. **Temporal data:** Follows the same pattern as location. No artifact-level date column exists. Temporal information appears only in assertions with predicates representing periods, phases, absolute dates, relative dates, or chronologies. These assertions may attach to artifacts directly or to related contexts, events, or observations. Multiple temporal assertions may coexist for a single artifact at different levels of the graph. Consolidating these into a single artifact-level date would require explicit semantic rules about which predicates matter and their precedence.
6. **Final state right now:** image—artifact mapping is understood, image download pipeline is running, artifact index is validated, location extraction kinda works but yields numeric coords only for ~2.5k artifacts, so that and date consolidation are the main missing pieces

Explicit low-level mapping of the three parquet files

oc_all_manifest.parquet (ENTITY REGISTRY)

Each row represents exactly one entity (node).

Core columns:

- `uuid`: bytes / hex (primary identifier)
- `item_type`: string (entity class)
- `label`: string (human-readable)
- `slug`: string (optional)
- `item_class_uuid`: UUID (typology / class)
- `project_uuid`: UUID (project association)
- `context_uuid`: UUID (parent context)
- `meta_json`: JSON (optional, sparse)

Important `item_type` values:

- `subjects` → artifacts
- `media` → conceptual images
- `projects` → projects / sites
- `predicates` → relation definitions
- `classes` → typologies
- `vocabularies`
- contexts (implicit via project / context UUIDs)

What does not live here:

- latitude / longitude
- dates
- image URLs
- file paths

oc_all_assertions.parquet (RELATIONS + VALUES)

Each row represents one directed semantic edge.

Core columns:

- `subject_uuid`: UUID (source entity)
- `predicate_uuid`: UUID (relation type)
- `object_uuid`: UUID (target entity, optional)
- `obj_double`: float (literal value)

- `obj_string`: string (literal value)
- `obj_integer`: int (literal value)
- `obj_datetime`: datetime (literal value)
- `meta_json`: JSON (optional)

Semantic interpretation:

`(subject_uuid) --[predicate_uuid]--> (object_uuid OR literal value)`

What lives here:

- artifact → media links
- artifact → context links
- artifact / context → location values
- artifact / context / event → temporal values
- all properties of artifacts

No semantic information exists outside this file.

`oc_all_resources.parquet` (PHYSICAL FILES)

Each row represents one physical file.

Core columns:

- `item_uuid`: UUID (points to media.uuid)
- `uri`: string (file URL)
- `sha256_checksum`: string
- `filesize`: int
- `created`: datetime
- `updated`: datetime

Key invariant:

`resource.item_uuid == media.uuid`

`resource.item_uuid ≠ artifact.uuid`

What lives here:

- images
- thumbnails
- previews
- other media files

Traversal paths (lowest level)

Artifact → Image:

```
manifest(subjects.uuid)
  → assertions.subject_uuid
    → assertions.object_uuid == manifest(media.uuid)
      → resources.item_uuid == manifest(media.uuid)
        → resources.uri
```

Artifact → Location:

```
manifest(subjects.uuid)
  → assertions.subject_uuid (direct)
    → obj_double / obj_string
```

OR

```
manifest(subjects.context_uuid)
  → assertions.subject_uuid (context)
    → obj_double / obj_string
```

Artifact → Date:

```
manifest(subjects.uuid OR subjects.context_uuid)
  → assertions.subject_uuid
    → obj_datetime / obj_string / obj_integer
```